

Take-Home Project: Optimize a 35B Model to Power a Coding Agent (open-claw) on a 16GB GPU

Format: Take-home (asynchronous), open-ended — Inference Optimization Engineer role

Expected Effort: ~1–2 days of focused work (budget extra for the model download and getting the stack running)

Submission: a private GitHub repo (invite evaluators) with your code/configs/logs + a structured write-up (Markdown, PDF, or doc)

Background

You have access to the following machine:

- Ubuntu 24.04
- NVIDIA RTX 5080 (16 GB VRAM)
- AMD Ryzen 9 9900X (12 cores / 24 threads)
- 32 GB system RAM

You are given an open-source coding agent, **open-claw** (provided with this assignment), and the model **Qwen3.6-35B-A3B**. Your goal is to make that model usable as **open-claw's backing model** on this machine — get it running, establish a baseline, then **iteratively optimize the inference stack**, measuring the impact **end-to-end through the agent** at every step.

If you don't have this exact machine, use a comparable one and treat **16 GB VRAM / 32 GB RAM** as the budget you are optimizing against (cap your usage accordingly; a cloud GPU instance is fine). Document your actual setup. Give yourself extra buffer if your machine is slow.

This project is intentionally open-ended and under-specified. There is no single correct solution.

The use case

open-claw is a **coding agent** (think Claude Code): it works a long-running task as a loop of many steps. On each step it receives a large, **mostly-static** context (system prompt, tool definitions, code/file context, running history) and produces a short, structured output (a tool call or a code edit) that is **verified** before it proceeds. A task can be tens of steps — latency-sensitive per step, but long-horizon, so cumulative wall-clock and reliability both matter.

The model is large (~**35B total params**, a Mixture-of-Experts with ~**3B active per token**) and your GPU is small (16 GB). Fitting and serving it well on this box is the heart of the exercise.

Focus

This is primarily an **inference-engine optimization** task, and the evaluation **must run through open-claw** — a working, agent-driven baseline is part of the deliverable, not a theoretical estimate. Spend your depth on the inference stack — quantization, the serving engine, how the model is placed across GPU/CPU/RAM, and how the agent's request pattern is served — rather than on changing the agent's logic. **Depth over breadth**: pick the bottlenecks that matter and go deep.

Phase 1 — Setup & Baseline

- Get **open-claw running locally** with **Qwen3.6-35B-A3B** as its backing model on the machine above. Document setup issues and how you resolved them — getting a hard stack actually running on a constrained box is part of the task.
- Establish a baseline by running a set of representative coding tasks **through open-claw** (at least 3 of varying complexity — see suggestions below, or draw from a popular agentic coding benchmark).
- Record baseline metrics and **document your measurement methodology** — what you measured, how, and why. Consider what actually matters for an agent: per-step latency, time-to-first-token on the large context, decode speed, whether tasks *complete correctly*, and total task wall-clock.

Phase 2 — Optimization Iterations (≥3)

Iteratively improve the inference stack. Choose the levers you believe matter most and go deep on a few (or propose your own):

- **Quantization** (e.g. GGUF, AWQ, GPTQ, bitsandbytes) — and how far you can push it before the agent starts failing tasks.
- **Serving / inference engine** (e.g. llama.cpp, vLLM, ktransformers, TensorRT-LLM, SGLang) — and how you place a 35B MoE across 16 GB VRAM + 32 GB RAM.
- **Serving the agent's request pattern** — the large, mostly-static context repeated across many steps.
- **Hardware-specific tuning** — CPU threads, memory mapping, offload split, batch sizing.

For each iteration:

- Target a **specific bottleneck**, and say how you identified it.
- Re-measure **through open-claw** using the same tasks as the baseline.
- Record before/after metrics — including the effect on **task success**, not just speed.

Phase 3 — Write-Up

- Environment and setup difficulties.
 - Baseline measurements + methodology, and why you chose them.
 - For each iteration: what you changed and why you expected it to help, the measured result, and what you learned (including negative results).
 - A summary table of all iterations (before/after).
 - The trade-offs you navigated (quality/reliability vs. speed vs. memory) and a recommendation tied to your definition of success.
 - What you'd pursue next with more time.
-

Problem understanding (address in the write-up)

- What clarification questions would you ask if your manager handed you this task?
- What assumptions did you make where those couldn't be answered?
- How did you define success — which metric(s) actually matter for a coding agent on this machine?

Suggested test tasks (drive these through open-claw)

Task	Description	Complexity
Simple	"Create a Python file that prints Hello World"	Low — single file, no reasoning
Medium	"Write a function that finds two numbers in a list that sum to a target"	Medium — algorithm design
Complex	"Refactor this function to handle edge cases and add unit tests" (provide a buggy snippet)	High — multi-step, multiple files

You may substitute or add tasks from a popular agentic coding benchmark; justify your choice.

Deliverables

- A **structured write-up** (as in Phase 3).
- **Code, configs, and scripts** for each iteration.
- **Raw performance logs / screenshots** showing before/after.
- A **README** explaining how to reproduce your setup.

Submit as a **private GitHub repository** (invite the evaluators). You'll walk us through your results and

your own contribution in the follow-up.

Expectations

We're not looking for a single "correct" answer. We're interested in how you:

- Break down an ambiguous problem
- Make and justify assumptions
- Get a hard system actually running, and design meaningful experiments
- Analyze results and identify bottlenecks
- Explain trade-offs
- Communicate your reasoning clearly

